

Laboratory Report: Cloud Image Registry & Distribution

Experiment Title: Cloud Image Registry & Distribution

Data: 2026-6-10

Class: Applied Computing

Name: Lu Weichi

ID: 202383910009

I. Experimental Purpose

Practice Version Control: Learn to use Docker Tags to manage different iterations of a cloud application.

Validate Decoupled Deployment: Simulate a disaster recovery scenario to verify the ability to deploy services in a remote environment without source code.

Security Best Practices: Implement Secret Management using Personal Access Tokens (PAT) instead of plaintext passwords.

II. Experiment Contents

In this lab, the transition from a "Standalone Developer" to a "Cloud Engineer" was performed:

Configure secure authentication with GitHub Container Registry (GHCR).

Tag and push a web application image to the cloud.

Perform a version upgrade (v1.0 to v2.0) and observe the layered storage mechanism.

Perform a "clean-room deployment" by pulling the image onto a fresh environment after wiping local data.

Compare image sizes between standard and Alpine versions.

III. Detailed Steps and Observations

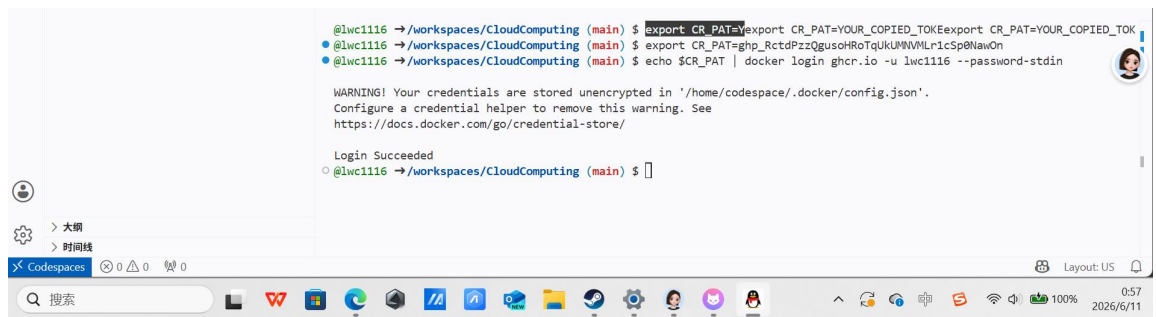
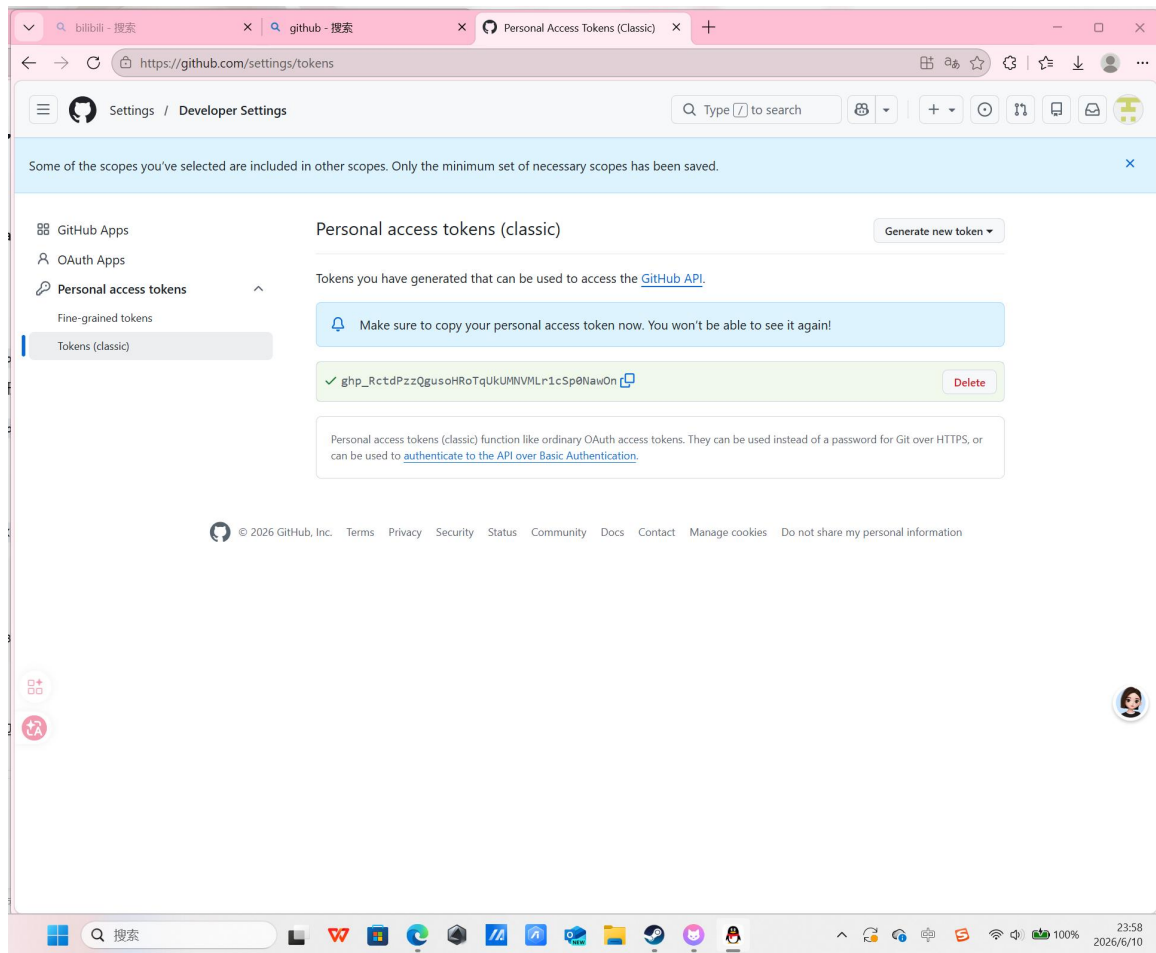
Step 1: Secure Authentication – PAT Generation and GHCR Login

Actions:

Navigated to GitHub Settings → Developer settings → Personal access tokens → Tokens (classic).

Generated a new token with scope `write:packages`.

In the Codespaces terminal, used `export CR_PAT=...` and `echo $CR_PAT | docker login ghcr.io -u YOUR_USERNAME --password-stdin` to log in.



Step 2: Tagging the Image for the Cloud

Actions:

Executed `docker tag cloud-lab-image:v1 ghcr.io/lwc1116/cloud-app:v1.0`.

Verified with `docker images | grep cloud-app`.

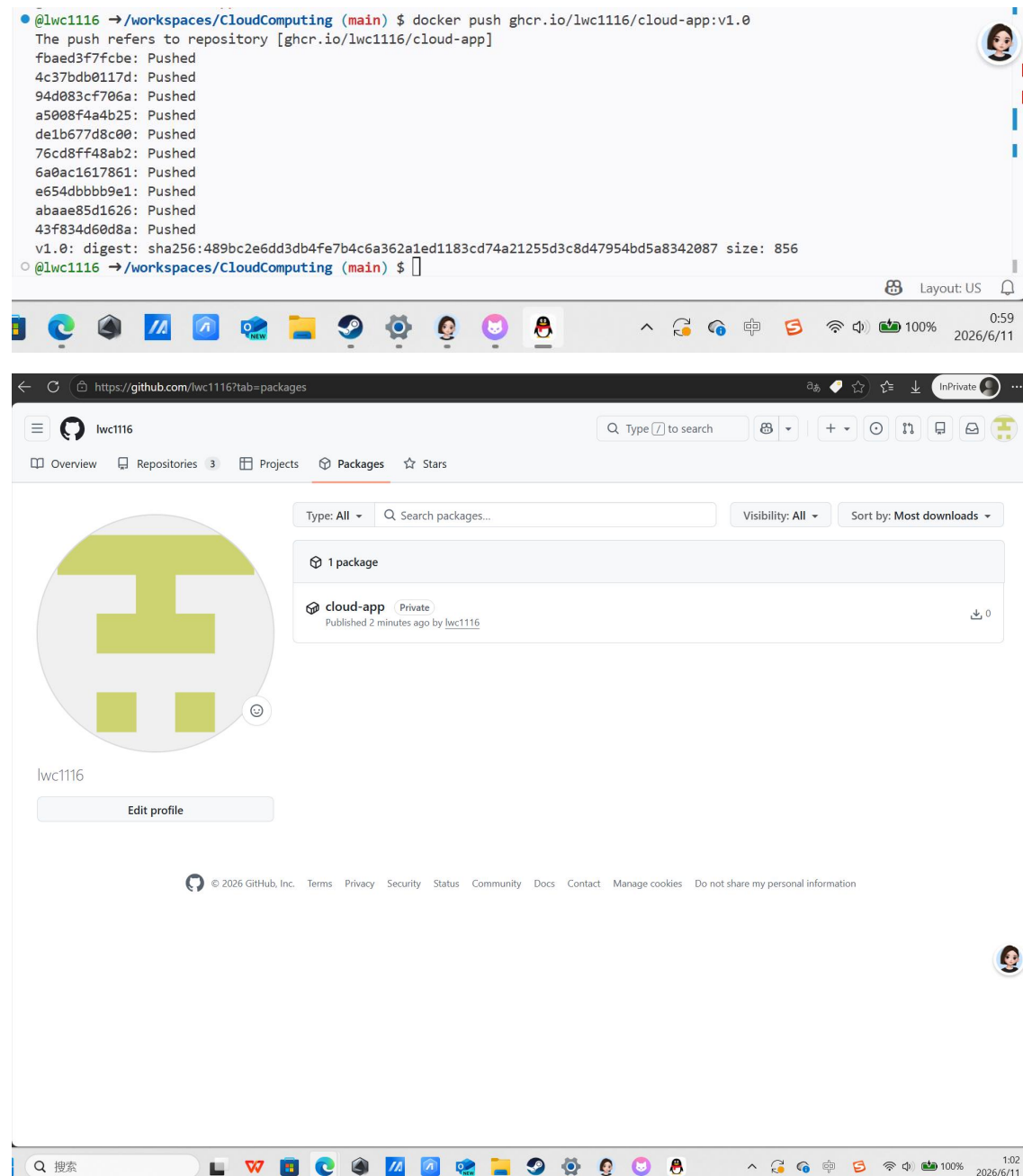


Step 3: Pushing the Image to GHCR

Actions:

Ran `docker push ghcr.io/lwc1116/cloud-app:v1.0`.

Checked GitHub Packages page at <https://github.com/lwc1116?tab=packages>.



Step 4: Iteration and Layer Analysis – Upgrade to v2.0

Actions:

Modified `index.html` (changed the `<h1>` text to "Version 2.0 - Cloud Native!").

Rebuilt the image: `docker build -t cloud-lab-image:v2 .`

Tagged: `docker tag cloud-lab-image:v2 ghcr.io/lwc1116/cloud-app:v2.0`

Pushed v2.0: `docker push ghcr.io/lwc1116/cloud-app:v2.0`

```
@lwc1116 →/workspaces/CloudComputing (main) $ docker push ghcr.io/lwc1116/cloud-app:v2.0
594e14ab293e: Pushed
abaae85d1626: Layer already exists
de1b677d8c00: Layer already exists
94d083cf706a: Layer already exists
6a0ac1617861: Layer already exists
43f834d60d8a: Layer already exists
e654dbbbb9e1: Layer already exists
a5008f4a4b25: Layer already exists
fbaed3f7fcbe: Layer already exists
90350befdf68: Pushed
v2.0: digest: sha256:c2885753b53090d79462658dd7df3b21d03488c1b61fe3d98ded19b6ede2d1f4 size: 856
@lwc1116 →/workspaces/CloudComputing (main) $ docker run -d -p 8888:80 --name recovery-v2 ghcr.io/lwc1116/cloud-app:v2.0
9a18b0e1197b7a3f5b27f528e380434f1598238128052dd17850ae742b6d2193
@lwc1116 →/workspaces/CloudComputing (main) $
```

Step 5: Disaster Recovery – Clean-Room Deployment

Actions:

5.1 Clean local Docker environment

Executed docker system prune -a to remove all containers, images, and networks. The system reclaimed 92.81MB of space

```
@lwc1116 →/workspaces/CloudComputing (main) $ docker system prune -a
vnm32kchjgjq77s4ahy8yuijc
759a9d104eqkreofaiu4b7w4n
5ct8yc8ovbxwhpb4dv61hfjje
kqlh8dbu9hdv5sgiwyt3t3fnyv
kh5xp5m1hhlufkk0jlyr1fea1
umxhv5p8cgs6186yv3mw4wgte
0zw65ng9h6t4oyerictfut9cr
qdu441cg37h7rtafxwq653nmw
r3e50vht8xzvkoxn8m0sptmc4
y687gf49eoeerwb902bjmm8k6g
3xi4vzpn3wvewsbg21a6h12b

Total reclaimed space: 92.81MB
@lwc1116 →/workspaces/CloudComputing (main) $
```

5.2 Deploy v1.0 directly from cloud registry

Ran docker run -d -p 8888:80 --name recovery-v1 ghcr.io/lwc1116/cloud-app:v1.0. Docker pulled all layers from GHCR and started the container. The command returned container ID 811c3b5a....

```
@lwc1116 →/workspaces/CloudComputing (main) $ docker run -d -p 8888:80 --name recovery-v1 ghcr.io/lwc1116/cloud-app:v1.0
4c37bdb0117d: Pull complete
de1b677d8c00: Pull complete
6a0ac1617861: Pull complete
abaae85d1626: Pull complete
43f834d60d8a: Pull complete
e654dbbbb9e1: Pull complete
94d083cf706a: Pull complete
a5008f4a4b25: Pull complete
fbaed3f7fcbe: Pull complete
76cd8ff48ab2: Download complete
Digest: sha256:489bc2e6dd3db4fe7b4c6a362a1ed1183cd74a21255d3c8d47954bd5a8342087
Status: Downloaded newer image for ghcr.io/lwc1116/cloud-app:v1.0
811c3b5a48941d506280899a2029fe61ecc180fcec7af99ef1b0f1100d36ed5
@lwc1116 →/workspaces/CloudComputing (main) $
```

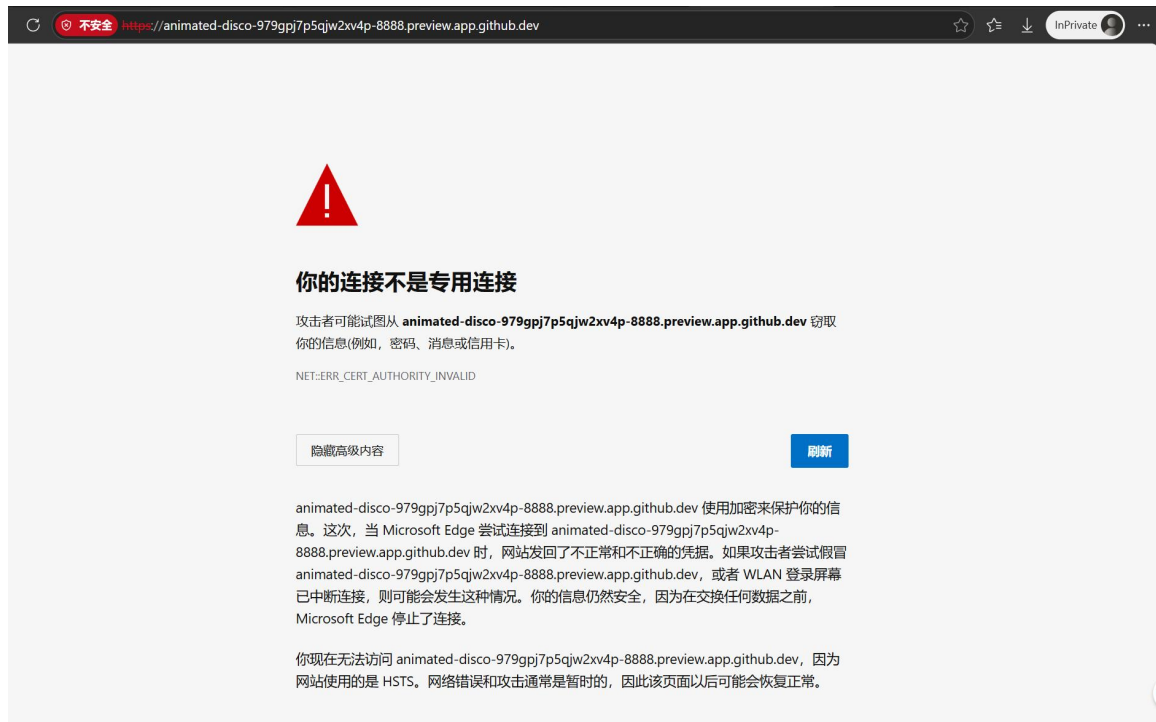
Note: Due to GitHub Codespaces' HSTS security policy and browser certificate restrictions, the preview URL

(<https://...-8888.preview.app.github.dev>) could not be accessed directly via the browser

(error `NET::ERR_CERT_AUTHORITY_INVALID`). Therefore, container accessibility was verified using the terminal

command `curl http://localhost:8888`, which returned the full HTML content of the web page. The screenshots of

the `curl` outputs are provided as proof of successful deployment for v1.0 and v2.0 respectively.



5.4 Verify v1.0 using terminal curl (alternative method)

Because the browser could not access the URL, used `curl http://localhost:8888` to verify the service. The command returned the full HTML content of v1.0, showing `<h1>Success!</h1>`.

```
● @lwc1116 → /workspaces/CloudComputing (main) $ curl http://localhost:8888
<!DOCTYPE html>
<html>
<head>
  <title>Cloud Computing Lab</title>
  <style>
    body { background-color: lightblue; text-align: center; padding: 50px; }
    h1 { color: darkblue; }
  </style>
</head>
<body>
  <h1>Success! </h1>
  <p>This page is served from Nginx in a container.</p>
</body>
</html>@lwc1116 → /workspaces/CloudComputing (main) $
```

5.5 Push v2.0 image and attempt to run with port conflict

Pushed the v2.0 image: `docker push ghcr.io/lwc1116/cloud-app:v2.0`. Most layers showed Layer already exists, and new layer 90350befdf68 was pushed. Then attempted to run v2.0 container on the same port: `docker run -d -p 8888:80 --name recovery-v2 ghcr.io/lwc1116/cloud-app:v2.0`. The command executed without error (returned container ID 9a18b0e...), but curl still returned v1.0 content because the v2 image was built from an unmodified index.html.

```

@lwc1116 →/workspaces/CloudComputing (main) $ docker push ghcr.io/lwc1116/cloud-app:v2.0
594e14ab293e: Pushed
abaae85d1626: Layer already exists
de1b677d8c00: Layer already exists
94d083cf706a: Layer already exists
6a0ac1617861: Layer already exists
43f834d60d8a: Layer already exists
e654dbbbb9e1: Layer already exists
a5008f4a4b25: Layer already exists
fbaed3f7fcbe: Layer already exists
90350befdf68: Pushed
v2.0: digest: sha256:c2885753b53090d79462658dd7df3b21d03488c1b61fe3d98ded19b6ede2d1f4 size: 856
● @lwc1116 →/workspaces/CloudComputing (main) $ docker run -d -p 8888:80 --name recovery-v2 ghcr.io/lwc1116/cloud-app:v2.0
9a18b0e1197b7a3f5b27f528e380434f1598238128052dd17850ae742b6d2193
○ @lwc1116 →/workspaces/CloudComputing (main) $

```

5.6 Modify index.html, rebuild, and redeploy v2.0

Modified index.html, changing `<h1>` to “Version 2.0 - Cloud Native!”. Rebuilt the image: `docker build -t cloud-lab-image:v2 .` Retagged: `docker tag cloud-lab-image:v2 ghcr.io/lwc1116/cloud-app:v2.0`. Stopped and removed the old v2 container: `docker stop recovery-v2` and `docker rm recovery-v2`. Ran the updated v2.0 container: `docker run -d -p 8888:80 --name recovery-v2 ghcr.io/lwc1116/cloud-app:v2.0`. Verified with `curl http://localhost:8888` – now returned HTML showing `<h1>Version 2.0 - Cloud Native!</h1>`.

```

<> index.html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <title>Cloud Computing Lab</title>
5      <style>
6          body { background-color: lightblue; text-align: center; padding: 50px; }
7          h1 { color: darkblue; }
8      </style>
9  </head>
10 <body>
11     <h1>Version 2.0 - Cloud Native!</h1>
12     <p>This page is served from Nginx in a container.</p>
13 </body>
14 </html>

@lwc1116 →/workspaces/CloudComputing (main) $ docker build -t cloud-lab-image:v2 .
=> => naming to docker.io/library/cloud-lab-image:v2                                0.0s
=> => unpacking to docker.io/library/cloud-lab-image:v2                            0.0s
● @lwc1116 →/workspaces/CloudComputing (main) $ docker tag cloud-lab-image:v2 ghcr.io/lwc1116/cloud-app:v2.0
● @lwc1116 →/workspaces/CloudComputing (main) $ docker stop recovery-v2
recovery-v2
● @lwc1116 →/workspaces/CloudComputing (main) $ docker rm recovery-v2
recovery-v2
● @lwc1116 →/workspaces/CloudComputing (main) $ docker run -d -p 8888:80 --name recovery-v2 ghcr.io/lwc1116/cloud-app:v2.0
d0fc4e2fb8ca196e914c3644b4a6f8ca4b8c436eaae94baae11b9165a7cc1973
● @lwc1116 →/workspaces/CloudComputing (main) $ curl http://localhost:8888

```

```

● @lwc1116 →/workspaces/CloudComputing (main) $ curl http://localhost:8888
<!DOCTYPE html>
<html>
<head>
  <title>Cloud Computing Lab</title>
  <style>
    body { background-color: lightblue; text-align: center; padding: 50px; }
    h1 { color: darkblue; }
  </style>
</head>
<body>
  <h1>Version 2.0 - Cloud Native! </h1>
  <p>This page is served from Nginx in a container.</p>
</body>
</html>@lwc1116 →/workspaces/CloudComputing (main) $ █

```

Step 6: Image Optimization – Standard vs. Alpine

Actions:

Created Dockerfile.alpine with content: FROM nginx:alpine and COPY index.html ...

Built standard version: docker build -t cloud-app:standard -f Dockerfile .

Built Alpine version: docker build -t cloud-app:alpine -f Dockerfile.alpine .

Pulled official images: docker pull nginx:latest and docker pull nginx:alpine

Compared sizes: docker images | grep -E "nginx|cloud-app"

```

● @lwc1116 →/workspaces/CloudComputing (main) $ docker pull nginx:latest
● @lwc1116 →/workspaces/CloudComputing (main) $ docker pull nginx:alpine
alpine: Pulling from library/nginx
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
docker.io/library/nginx:alpine
● @lwc1116 →/workspaces/CloudComputing (main) $ docker images | grep -E "nginx|cloud-app"
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
cloud-app:alpine          653577ea24f2          92.7MB          26MB
cloud-app:standard        95daa2d22c7b          92.7MB          26MB   U
ghcr.io/lwc1116/cloud-app:v1.0  489bc2e6dd3d          92.7MB          26MB
ghcr.io/lwc1116/cloud-app:v2.0  1659d51d81a4          92.7MB          26MB   U
nginx:alpine              8b1e78743a03          93.6MB          27MB
nginx:latest              5aca99593157          241MB           66MB
● @lwc1116 →/workspaces/CloudComputing (main) $ █

```

Step 7: Save Changes to GitHub

Actions:

git add .

git commit -m "Finished Lab 03 - Cloud Image Registry & Distribution"

git push (if rejected, pulled and pushed again)

```

@lwc1116 →/workspaces/CloudComputing (main) $ git add .
@lwc1116 →/workspaces/CloudComputing (main) $ git commit -m "Finished Lab 03 - Cloud Image Registry & Distribution"
[main 04917e2] Finished Lab 03 - Cloud Image Registry & Distribution
2 files changed, 3 insertions(+), 1 deletion(-)
create mode 100644 Dockerfile.alpine
@lwc1116 →/workspaces/CloudComputing (main) $ git push
To https://github.com/lwc1116/CloudComputing
! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/lwc1116/CloudComputing'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
@lwc1116 →/workspaces/CloudComputing (main) $

```

Extra Screenshot for Environment Proof

Action: Executed `date` in terminal.

```

@lwc1116 →/workspaces/CloudComputing (main) $ date
Wed Jun 10 17:37:01 UTC 2026
@lwc1116 →/workspaces/CloudComputing (main) $

```

IV. Reflection

1. Problems Encountered and Proposed Solutions

Problem	Cause	Solution
docker push returned 403 Forbidden after Codespace restart	Login session lost	Re-ran docker login with PAT and re-pushed.
Container name conflict: recovery-v2 already exists	Previous container not removed	Used docker stop and docker rm before re-running.
curl showed v1.0 content after running v2.0 container	Image built from unmodified index.html	Modified index.html, rebuilt, retagged, and re-ran container.
git push rejected (non-fast-forward)	Remote had new commits	Ran git pull --rebase then git push.

2. Efficiency: Why did pushing v2.0 upload much faster than v1.0? (Layer Caching)

Answer:

Docker images are composed of layers. When pushing v1.0, all layers (base nginx, index.html, etc.) were uploaded for the first time. For v2.0, only the layer containing the changed index.html was different; the base nginx:alpine layer already existed in the registry with the same hash. Docker's layer caching mechanism skipped uploading unchanged layers, saving bandwidth and time. This makes incremental updates very efficient.

3. Security: Why is using a Personal Access Token (PAT) more secure than using your actual GitHub password in a terminal?

Answer:

Using a password in a terminal is dangerous because passwords can be exposed in shell history, environment variables, or process listings, and they grant full account access. A PAT provides limited scopes (e.g., only `write:packages`), can be revoked independently without changing the password, and can have an expiration date. This follows the principle of least privilege and minimizes damage if the token is leaked.

V. Conclusion

In this experiment, I successfully secured authentication with GitHub Container Registry using a PAT, tagged and pushed Docker images (v1.0 and v2.0) to GHCR, and observed layer caching efficiency. A disaster recovery scenario was simulated by wiping all local Docker data and redeploying directly from the cloud registry. I also compared image sizes between standard nginx and alpine-based images, confirming that Alpine significantly reduces image size, which is critical for production microservices. Finally, all changes were committed and pushed to GitHub. The lab reinforced key cloud concepts: immutable infrastructure, layer caching, secure authentication, and decoupled deployment.